DSST289: Introduction to Data Science

Taylor Arnold
University of Richmond
tarnold2@richmond.edu

Fall 2026

Course Notes

Outline

Welcome!

This is the first semester of our year-long course in data science. Our goal of the two semesters is to introduce the core techniques of data science and how to apply them with a programming language.

The **first semester** is designed to focus on data creation, visualization, and manipulation.

The **second semester** moves towards working with increasingly complex data types such as spatial, temporal, network, text and image data. We will also continue to practice and become more comfortable with the core techniques throughout.

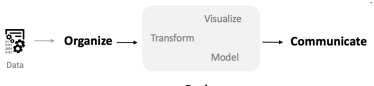
Conceptual visualization of the data science pipeline.

Instructors

This course is co-taught by Prof. Taylor Arnold (`taylor.arnold@richmond.edu`) and Prof. Sam Kellogg (`sam.kellogg@richmond.edu`).

Both of us will be in class and either of us is happy to hear from you about anything to do with the course material, the readings, the homework, or the project.

Prof. Arnold is the instructor of record, so please direct any questions about grades to him.



Conceptual visualization of the data science pipeline.

Format

The semester is broken into three units, each ending with an in-class exam, followed by a final project that runs through the end of the term. Class meetings are hands-on and interactive. Please bring a computer, pencil, and something to write on to each class. Class materials can be found on the course website.

All of the material is available on the course website: `https://taylor-arnold.github.io/courses/dsst289-s26/`

DSST289: Introduction to Data Science

[syllabus] [dan] [book] [solutions] [sheet] [slides] [zoom] [form] [poll]

Below are the readings and homework to be done before each class. There are also links to the notebooks we will be working on together in class. Study guides for the exams will be posted below at least one week ahead of time. The dates of the external talks, of which you must attend two, are given at the bottom of this page.

A **final project** will be introducted in class on October 26th. You will design, collect, document, and analyze a dataset about your own life over a fixed fourteen-day window running from **November 2nd through November 15th**. Read the project description before Homework 15. There is no final exam.

Date	Topics	Reading	Homework	Notebook
2026-08-24	Course Introduction	—	—	—
2026-08-26	Introduction to Python	1.1–1.7	Homework01	Notebook01
2026-08-31	Tabular Data	1.8–1.10	Homework02	Notebook02
2026-09-02	Organize: Sort + Select	2.1–2.6	Homework03	Notebook03
2026-09-07	Organize: Rename + Modify	2.7–2.11	Homework04	Notebook04
2026-09-09	Data Types	3.1–3.9	Homework05	Notebook05
2026-09-14	[Exam I]			
2026-09-16	Grammar of Graphics	4.1–4.3	Homework06	Notebook06
2026-09-21	Aesthetics + Geometries	4.4–4.8	Homework07	Notebook07

Screen shot from the course website. Please check website for most up-to-date details.

Class Structure

The general structure of each class:

- (Before class) Do the assigned reading
- (Before class) Do the assigned homework (hand-written)
- (Start of class) Fill out class form for attendence and homework completion
- Slides for new material (5-10 minutes)
- Discussion of the homework in groups (5-10 minutes)
- Questions and/or class poll of new material (5 minutes)
- Review of last notebook (5)
- Work on current class notebook (remainder of the class, roughly ~45 minutes)

DSST289: Introduction to Data Science

[syllabus] [dan] [book] [solutions] [sheet] [slides] [zoom] [form] [poll]

Below are the readings and homework to be done before each class. There are also links to the notebooks we will be working on together in class. Study guides for the exams will be posted below at least one week ahead of time. The dates of the external talks, of which you must attend two, are given at the bottom of this page.

A **final project** will be introducted in class on October 26th. You will design, collect, document, and analyze a dataset about your own life over a fixed fourteen-day window running from **November 2nd through November 15th**. Read the project description before Homework 15. There is no final exam.

Date	Topics	Reading	Homework	Notebook
2026-08-24	Course Introduction	—	—	—
2026-08-26	Introduction to Python	1.1–1.7	Homework01	Notebook01
2026-08-31	Tabular Data	1.8–1.10	Homework02	Notebook02
2026-09-02	Organize: Sort + Select	2.1–2.6	Homework03	Notebook03
2026-09-07	Organize: Rename + Modify	2.7–2.11	Homework04	Notebook04
2026-09-09	Data Types	3.1–3.9	Homework05	Notebook05
2026-09-14	[Exam I]			
2026-09-16	Grammar of Graphics	4.1–4.3	Homework06	Notebook06
2026-09-21	Aesthetics + Geometries	4.4–4.8	Homework07	Notebook07

Screen shot from the course website. Please check website for most up-to-date details.

Grading

The final grade is an evenly weight blend of five components:

- Exam I (20%)
- Exam II (20%)
- Exam III (20%)
- Final Project (20%)
- Attendance + Homework (20%)

The exams will be given in class on the dates listed. Full study guides are posted on the website.

DSST289: Introduction to Data Science

[syllabus] [dan] [book] [solutions] [sheet] [slides] [zoom] [form] [poll]

Below are the readings and homework to be done before each class. There are also links to the notebooks we will be working on together in class. Study guides for the exams will be posted below at least one week ahead of time. The dates of the external talks, of which you must attend two, are given at the bottom of this page.

A **final project** will be introducted in class on October 26th. You will design, collect, document, and analyze a dataset about your own life over a fixed fourteen-day window running from **November 2nd through November 15th**. Read the project description before Homework 15. There is no final exam.

Date	Topics	Reading	Homework	Notebook
2026-08-24	Course Introduction	—	—	—
2026-08-26	Introduction to Python	1.1–1.7	Homework01	Notebook01
2026-08-31	Tabular Data	1.8–1.10	Homework02	Notebook02
2026-09-02	Organize: Sort + Select	2.1–2.6	Homework03	Notebook03
2026-09-07	Organize: Rename + Modify	2.7–2.11	Homework04	Notebook04
2026-09-09	Data Types	3.1–3.9	Homework05	Notebook05
2026-09-14	[Exam I]			
2026-09-16	Grammar of Graphics	4.1–4.3	Homework06	Notebook06
2026-09-21	Aesthetics + Geometries	4.4–4.8	Homework07	Notebook07

Screen shot from the course website. Please check website for most up-to-date details.

External Speakers

You are required to attend at least two external talks over the course of the semester. These are a required component of the course but are not associated with a numerical grade. A list of talks is posted on the course website; any of those will count with not explaination needed.

You may also count any other on-campus talk with a data science component. For a talk that is not on our list, you will be asked to write a short justification explaining what the data science component was. Attendance is recorded through the talk form linked on the course website.

DSST289: Introduction to Data Science

[syllabus] [dan] [book] [solutions] [sheet] [slides] [zoom] [form] [poll]

Below are the readings and homework to be done before each class. There are also links to the notebooks we will be working on together in class. Study guides for the exams will be posted below at least one week ahead of time. The dates of the external talks, of which you must attend two, are given at the bottom of this page.

A **final project** will be introducted in class on October 26th. You will design, collect, document, and analyze a dataset about your own life over a fixed fourteen-day window running from **November 2nd through November 15th**. Read the project description before Homework 15. There is no final exam.

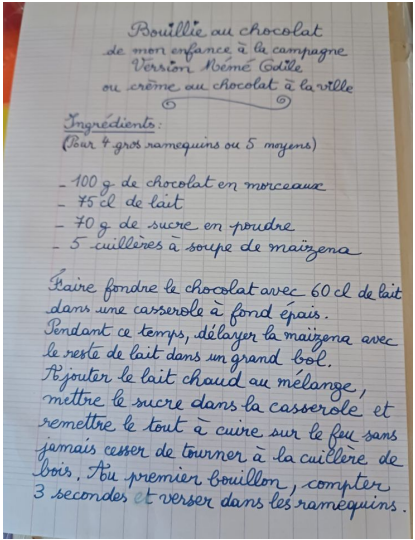
Date	Topics	Reading	Homework	Notebook
2026-08-24	Course Introduction	—	—	—
2026-08-26	Introduction to Python	1.1–1.7	Homework01	Notebook01
2026-08-31	Tabular Data	1.8–1.10	Homework02	Notebook02
2026-09-02	Organize: Sort + Select	2.1–2.6	Homework03	Notebook03
2026-09-07	Organize: Rename + Modify	2.7–2.11	Homework04	Notebook04
2026-09-09	Data Types	3.1–3.9	Homework05	Notebook05
2026-09-14	[Exam 1]			
2026-09-16	Grammar of Graphics	4.1–4.3	Homework06	Notebook06
2026-09-21	Aesthetics + Geometries	4.4–4.8	Homework07	Notebook07

Screen shot from the course website. Please check website for most up-to-date details.

Recipe

Let's do a little data work today to start thinking about the core questions of data science.

On a piece of paper, write down a recipe for a favorite dish of yours. Something moderately complicated (around 5-10 ingredients) is perfect. Include the ingredients and instructions for how to make it.


Bouillie au chocolat
de mon enfance à la campagne
Version Noémé Cahle
ou crème au chocolat à la ville

Ingrédients :
(Pour 4 gros ramequins ou 5 moyens)

- 100 g de chocolat en morceaux
- 75 cl de lait
- 70 g de sucre en poudre
- 5 cuillères à soupe de maïzena

Faire fondre le chocolat avec 60 cl de lait dans une casserole à fond épais.
Pendant ce temps, délayer la maïzena avec le reste de lait dans un grand bol.
Ajouter le lait chaud au mélange, mettre le sucre dans la casserole et remettre le tout à cuire sur le feu sans jamais cesser de tourner à la cuillère de bois. Au premier bouillon, compter 3 secondes et verser dans les ramequins.

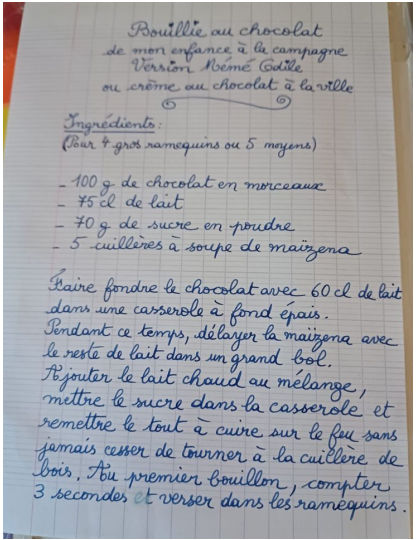
Example of a hand-written recipe.

Recipe

As you read the section for next class, consider what it would take to create a tabular version of your recipe as a structured dataset. Consider how we could create a large dataset of hundreds or thousands of these.

What if we needed a way to do things such as detect recipes that are vegetarian, vegan, kosher, gluten free? What if we wanted to calculate how to scale up/down you version? Convert from/to metric? Figure out what things you could make with a certain ingredient?

Think about all of these things as a concrete example as you read for Wednesday's class!



Bouillie au chocolat
de mon enfance à la campagne
Version Noémé Cahile
ou crème au chocolat à la ville

Ingrédients :
(Pour 4 gros ramequins ou 5 moyens)

- 100 g de chocolat en morceaux
- 75 cl de lait
- 70 g de sucre en poudre
- 5 cuillères à soupe de maïzena

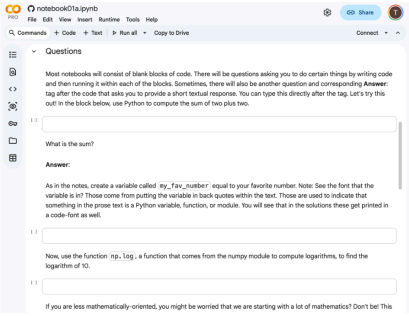
Faire fondre le chocolat avec 60 cl de lait dans une casserole à fond épais.
Pendant ce temps, délayer la maïzena avec le reste de lait dans un grand bol.
Rajouter le lait chaud au mélange, mettre le sucre dans la casserole et remettre le tout à cuire sur le feu sans jamais cesser de tourner à la cuillère de bois. Au premier bouillon, compter 3 secondes et verser dans les ramequins.

Example of a hand-written recipe.

Google Colab

We will use the Google Colab environment for running Python. This allows us to run Python code directly from the browser for free using your university Google account with zero setup required.

Please keep in mind that Colab is simply the means of running Python. Everything we learn is directly applicable to running Python on your own machine or using a different cloud-based service.



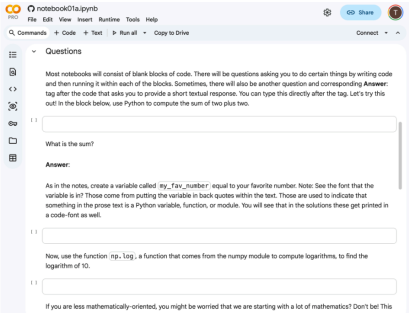
The Google Colab environment running in a web browser.

Running Code

Code in Colab is organized into **cells**. There are two ways to run the code inside a cell:

- Click the **run button** (the circle with a triangle) on the left side of the cell.
- Use a **keyboard shortcut**: `Shift+Enter` runs the cell and moves to the next one; `Ctrl+Enter` (`Cmd+Enter` on a Mac) runs the cell in place.

The keyboard shortcuts are much faster once you get used to them, so try to make a habit of using them.

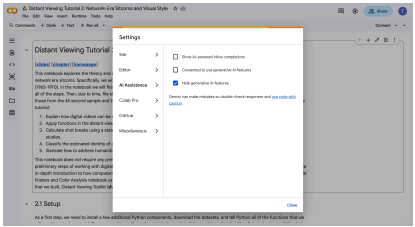


Hover over a cell to reveal the run button on its left side.

Colab Settings

Before we start, please change two sets of settings. Open the **Tools > Settings** dialog and:

1. Go to the **AI Assistance** tab. Turn off the first two options and set the third to **hide generative AI**, as in the image.
2. Go to the **Editor** tab and unselect **“Automatically close brackets and quotes in code cells”**.



The AI Assistance settings, configured as required.

Outline

Exploratory Data Analysis

We will focus on tools and techniques for **exploratory data analysis**, or EDA.

- Introduced by John Tukey in his 1977 book of the same name
- Examines data through visualizations and broad summary statistics
- Prioritizes studying data directly to generate hypotheses and spot trends, prior to (or instead of) formal statistical modeling

EDA is now widely used across statistics, computer science, and many other data-driven fields.

The Python Data Science Ecosystem

The ideas of EDA have been adopted and extended in Python through a rich ecosystem of libraries:

- **pandas** (Wes McKinney, 2008): brought R-like data structures to Python
- **matplotlib** / **seaborn**: general-purpose plotting
- **plotnine**: grammar-of-graphics plotting, following R's ggplot2
- **Polars**: a newer, high-performance alternative to pandas

In this course, we will build our workflow around **Polars** and **plotnine**.

Working with Notebooks

We will write and run our code in **Quarto notebooks** (`.qmd` files), which let us mix code, output, and explanatory text in one document.

- A notebook is organized into *cells*: code cells or text (markdown) cells
- Running a code cell executes the Python code and shows the output directly below it
- Code cells can be run with the run button or `Shift+Enter`
- Notebooks can be run for free in **Google Colab**, with no local setup required

Python as a Calculator

In its simplest form, Python acts as a calculator.

Running a code cell displays its output directly beneath it.

```
1 + 1
# output: 2
```

Variables

Running code can also store values in *variables*, using the = symbol.

The variable name goes on the left; the expression that produces its value goes on the right.

Variable names should use only lowercase letters, numbers, and underscores.

```
mynum = 3 + 5
# (no output; value is saved)

mynum + 1
# output: 9
```

Functions

A *function* takes a set of input values (arguments) and returns an output value.

For example, `round()` rounds a number to the nearest integer, or to a set number of digits.

```
round(3.14159)
# output: 3

round(3.14159, 2)
# output: 3.14
```

Named Arguments

Arguments can also be passed by *name* rather than by position, which often makes code easier to read.

To learn a function's arguments, use `help()` or consult the module's online documentation.

```
round(number=3.14159, ndigits=2)
# output: 3.14

help(round)
```

Importing Modules

Lines starting with `import` make a module available; access its functions with `module.function()`.

`from ... import` brings specific functions in directly, so no prefix is needed.

The `as` clause creates a shorter alias for typing convenience.

```
import polars as pl
from plotnine import ggplot, aes
from polars import col as c

import funs
```

Loading Data

Tabular data is often stored in plain-text CSV files.

`pl.read_csv()` reads a CSV file into a Polars *DataFrame*, given a path relative to the script's location.

Each row is an *observation*; each column is a *variable*.

```
country = pl.read_csv("data/countries.csv")
country
# shape: (135, 15)
```

Formatting Code

Consistent formatting (following PEP 8) makes code easier to read, debug, and maintain:

- snake_case names, spaces around operators
- double-quoted strings
- long expressions wrapped in parentheses
- one method or operator per line when chaining

```
(
    country
    .filter(c.region == "Europe")
    .pipe(ggplot, aes(c.hdi, c.happy))
    .geom_point()
    .geom_text(aes(label=c.iso), size=6, nudge_y=0.5)
)
```

Outline

Organizing a Dataset

This chapter covers the core methods for reshaping a single DataFrame: reordering rows, choosing columns, filtering rows, creating new columns, and removing duplicates.

- Every method here works on one DataFrame at a time
- Each takes a DataFrame as input and returns a new DataFrame, so methods can be chained
- The original DataFrame is never modified; save a result to keep it

Referring to Columns

Most methods need to be told which column to work on, using the `c.` prefix.

Attach a comparison, a method, or arithmetic to build a *recipe*: Polars works through it once per row.

```
c.pop
c.hdi > 0.9
c.pop.sqrt()
c.pop * 1000
```

Sorting Rows

`.sort()` reorders rows by one or more columns.

Set `descending=True` to reverse order.

With several columns, ties in the first are broken by the next.

```
(
    country
    .sort(c.pop, descending=True)
)

(
    country
    .sort(c.region, c.pop)
)
```

Selecting Columns

`.select()` returns only the listed columns, in the order given.

`.drop()` does the opposite: it removes the listed columns and keeps the rest.

```
(
    country
    .select(c.iso, c.lat, c.lon)
)

(
    country
    .drop(c.lat, c.lon)
)
```

Selecting Rows by Position

`.head(n=)` returns the first `n` rows; `.tail(n=)` returns the last `n`.

`.sample()` randomly selects rows: give `n=` for a count or `fraction=` for a proportion.

```
(
    country
    .head(n=4)
)
```

```
(
    country
    .sample(fraction=0.2)
)
```

Filtering Rows

`.filter()` keeps rows where a condition evaluates to `True`.

Rows where the tested value is missing are dropped as well.

Use == for equality; a single = is reserved for assignment.

```
(
    country
    .filter(c.hdi > 0.9)
)

(
    country
    .filter(c.region == "Europe")
)
```

Combining Conditions

Combine conditions with `&` (and) and `|` (or); wrap each one in parentheses.

`.is_in()` checks membership in a set of values.

```
(
    country
    .filter(
        (c.region == "Europe") & (c.hdi > 0.9)
    )
)

(
    country
    .filter(c.region.is_in(["Europe", "Africa"]))
)
```

Rename and Unique

`.rename()` takes a dictionary mapping old names to new ones.

`.unique()` drops duplicate rows; pass `subset=` to compare only certain columns.

```
(
    country
    .rename({"pop": "population"})
)
```

```
(
    country
    .unique(subset=["region"])
)
```

```
shape: (5, 15)
region      iso    full_name    pop    ...
"Americas"  "VEN"  "Venezuela"  28.52  ...
"Africa"    "SEN"  "Senegal"    18.93  ...
"Oceania"   "AUS"  "Australia"  26.97  ...
"Asia"      "LKA"  "Sri Lanka"  23.23  ...
"Europe"    "FIN"  "Finland"    5.623  ...
```

Modifying Columns

`.with_columns()` adds or overwrites columns, keeping the rest of the DataFrame.

Name each new column with =; attach a method like `.sqrt()` for common transformations.

```
(
    country
    .with_columns(
        population_1k = c.pop * 1000,
        pop_sqrt = c.pop.sqrt()
    )
)
```

Pipes

`df.pipe(f)` does the same thing as `f(df)`, but keeps it as one more step in a chain.

Extra arguments to `.pipe()` are passed straight through to the function.

```
(
    country
    .pipe(type)
)
```

```
(
    country
    .print(4)
)
```

```
<class 'polars.dataframe.frame.DataFrame'>

shape: (135, 15)
iso    full_name   region     pop    ...
"SEN"  "Senegal"   "Africa"   18.93  ...
"VEN"  "Venezuela" "Americas" 28.52  ...
...    ...         ...        ...    ...
"CYP"  "Cyprus"    "Asia"     1.371  ...
"PAK"  "Pakistan"  "Asia"     255.2  ...
```

Conditional Columns

`pl.when()`, `.then()`, and `.otherwise()` build a column from a condition.

Chain more `.when().then()` pairs for several conditions; the first match wins.

Wrap text values in `pl.lit()` so they aren't read as column names.

```
(
    country
    .with_columns(
        zone = (
            pl.when(c.lat.abs() < 23.5)
              .then(pl.lit("tropical"))
            .otherwise(pl.lit("temperate"))
        )
    )
)
```

Outline

Data Types

Every column in a DataFrame has exactly one *type*, which determines whether its values can be added together, whether they sort numerically or alphabetically, and how missing entries behave.

- Four types cover almost everything in this course: `str`, `i64`, `f64`, `bool`
- `.schema` lists every column's type at once
- A DataFrame read from a messy file can have the wrong type without any error being raised

Checking the Schema

Printing a DataFrame shows each column's type directly under its name.

`.schema` lists every column name alongside its type in one place.

```
country.schema
# Schema({'iso': String, 'pop': Float64,
#         'hdi': Float64, ...})
```

The Four Core Types

`str` holds text, `i64` holds whole numbers, `f64` holds numbers with a decimal part.

Comparison operators (>, <, ==, `!=`) always build a `bool` column, whatever the type of their inputs.

```
(
    country
    .select(c.iso, c.hdi, high_hdi = c.hdi > 0.9)
)
# high_hdi column has type bool
```

A Column with the Wrong Type

A messy file can silently mis-type a column: ZIP codes read as integers lose leading zeros, and one stray `"n/a"` forces an entire numeric column to become `str`.

Strings compare alphabetically, so `.max()` on a broken numeric column gives nonsense, and arithmetic on it raises an error outright.

```
cities.schema
# {'zip': Int64, 'pop': String, ...}

cities.select(c.pop.max())
# output: "n/a"
```

Casting Columns

`.cast()` converts a column to a new type, named with a `pl.` prefix: `pl.Int64`, `pl.Float64`, `pl.String`, `pl.Boolean`.

Integer to float is always safe. Float to integer truncates the decimal part rather than rounding.

```
(
    country
    .select(c.iso, c.lat, lat_int = c.lat.cast(pl.Int64))
)
# 41.9 -> 41,  -34.6 -> -34
```

Casting Invalid Values

A plain cast raises an error if any value cannot be converted to the target type.

`strict=False` converts anyway, replacing unconvertible values with a missing value.

```
(
    cities
    .select(
        c.city,
        pop = c.pop.cast(pl.Int64, strict=False)
    )
)
# "n/a" -> null
```

Missing vs. Invalid Values

`null` marks a missing value; any column of any type can contain one.

`NaN` (”not a number”) is different: it is the result of a computation with no valid answer, such as the square root of a negative number.

```
(
    country
    .select(c.iso, c.lat, lat_sqrt = c.lat.sqrt())
)
# negative lat -> NaN
```

Testing and Filling

`.is_null()` / `.is_nan()` test for each kind of problem; `.fill_null()` / `.fill_nan()` replace each with a chosen value.

`.drop_nulls()` and `.drop_nans()` remove the corresponding rows; dropping one never touches the other.

```
(
    cities
    .select(
        c.city,
        pop = c.pop.cast(pl.Int64, strict=False)
                   .fill_null(0)
    )
)
```

Null Propagation

Arithmetic and comparisons involving a null produce another null, not an error or a guess.

`.filter()` drops rows where the test comes out null, alongside the `False` ones.

Aggregations like `.sum()` and `.mean()` are the exception: they skip nulls instead of propagating them.

```
c.pop > 100
# missing pop -> null, not False
```

Expressions

A `c.` fragment such as `c.pop` is not data. It is an *expression*: a description of a computation, with no DataFrame attached.

Arithmetic, comparisons, and methods build one expression on top of another.

```
c.pop
# description, not values

c.pop.sqrt()
c.hdi > 0.9
```

Expressions and Contexts

An expression only produces values once it is handed to a *context*: a method that evaluates it against real data.

`.select()` and `.with_columns()` evaluate expressions into columns; `.filter()` keeps rows where the result is `True`; `.sort()` uses it to order rows.

The same expression moves unchanged between contexts, which is why `c.` keeps reappearing for the rest of the course.

```
country.select(high_hdi = c.hdi > 0.9)
country.filter(c.hdi > 0.9)
# same expression, different context
```

Strings and Dates

String columns carry methods under `.str`, for tasks like changing case or extracting text.

`.str.to_date()` parses text into a genuine `date` column, which supports calendar-aware arithmetic.

```
country.select(
    c.iso,
    name_upper = c.full_name.str.to_uppercase()
)
```

```
cities.select(c.city, founded = c.founded.str.to_date())
```

```
shape: (8, 2)
city           founded
str            date
"Agawam"       1855-05-17
"Amherst"      1759-02-13
"Boston"       1630-09-07
"Lowell"       1826-03-01
"Pittsfield"   1761-04-21
"Salem"        1626-06-24
"Springfield"  1636-05-14
"Worcester"    1722-06-14
```

Outline

The Grammar of Graphics

Plots in this course use **plotnine**, built around the *grammar of graphics*: a visualization is assembled from independent layers.

- *Data*: the DataFrame the plot draws from
- *Aesthetics* (`aes()`): map columns to visual properties such as position, color, or size
- *Geometry*: the kind of mark drawn per row — points, text, lines, bars

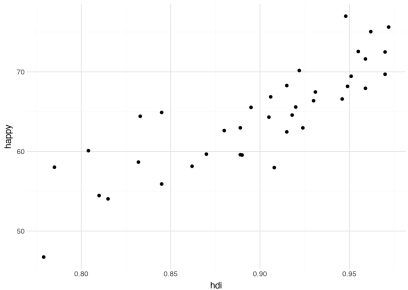
Every plot starts the same way: pipe a DataFrame into `ggplot`.

Points Geometry

Every plotnine chart begins the same way: pipe a DataFrame into `ggplot`, describe the `x` and `y` aesthetics, and add a geometry.

A *point geometry* (`geom_point()`) draws one point per row. Below, each point is a European country, placed by its HDI and happiness score.

Countries with higher human development tend to report higher happiness.



Scatter plot of HDI against happiness for European countries, one point per row.

Building a Scatter Plot

`.pipe(ggplot, aes(x, y))` starts a plot, mapping columns to the axes.

Chain on a geometry method: `.geom_point()` draws one point per row.

```
(
    country
    .filter(c.region == "Europe")
    .pipe(ggplot, aes(c.hdi, c.happy))
    .geom_point()
)
```

Optional Aesthetics

Beyond `x` and `y`, most geometries also accept `color`, `fill`, `alpha`, `size`, and `shape`.

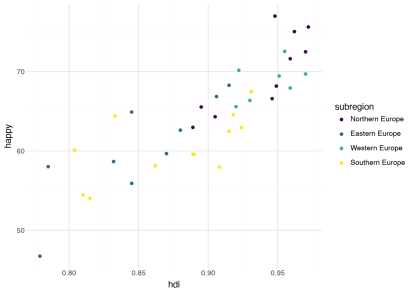
A variable mapping goes inside a second `aes()` call, passed to the geometry.

```
(
    country
    .filter(c.region == "Europe")
    .pipe(ggplot, aes(c.hdi, c.happy))
    .geom_point(aes(color=c.subregion))
)
```

Mapping a Column to Color

Mapping `subregion` to `color` adds a third variable to the plot without changing what the axes show.

plotnine picks the colors automatically and builds the legend on the right.



The same scatter plot, colored by subregion.

Fixed vs. Mapped Aesthetics

A *fixed* aesthetic — the same value for every row — goes outside `aes()`, directly in the geometry call.

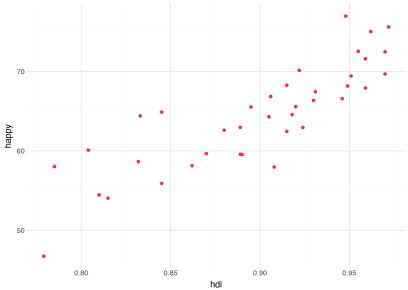
A *mapped* aesthetic — one that varies by row — goes inside `aes()`.

```
(
    country
    .filter(c.region == "Europe")
    .pipe(ggplot, aes(c.hdi, c.happy))
    .geom_point(color="#F5276C")
)
```

One Fixed Color for Every Point

Every point is now the same pink, set once outside `aes()`, instead of one color per `subregion`.

Compare with the colored version: the positions haven't changed, only how the points are drawn.



The same scatter plot with a single fixed point color.

Text Geometry

`geom_text()` draws a text label per row instead of a point.

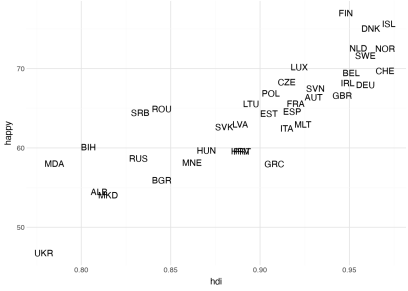
It requires a `label` aesthetic, in addition to `x` and `y`.

```
(
    country
    .filter(c.region == "Europe")
    .pipe(ggplot, aes(c.hdi, c.happy))
    .geom_text(aes(label=c.iso))
)
```

Labels Instead of Points

Each country's ISO code sits where its point would have been, positioned by the same `hdi` and `happy` aesthetics.

Labels identify every country at once, at the cost of overlapping text where points are close together.



Country codes plotted by HDI and happiness, with no points underneath.

Layering Geometries

Geometries stack: chain `.geom_point()` and `.geom_text()` to draw both in one plot.

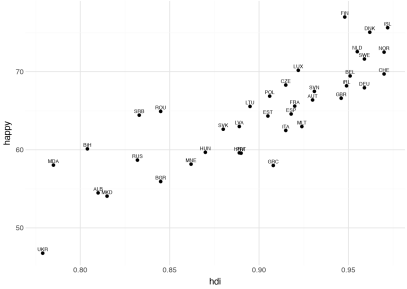
`nudge_y` and `size` move and shrink the labels so they don't sit on top of the points.

```
(
    country
    .filter(c.region == "Europe")
    .pipe(ggplot, aes(c.hdi, c.happy))
    .geom_point()
    .geom_text(aes(label=c.iso), size=6, nudge_y=0.5)
)
```

Points and Labels Together

Points mark the exact HDI and happiness values; labels identify which country each point belongs to.

`nudge_y` shifts each label above its point so the two don't overlap.



Points and nudged text labels layered in a single plot.

Line Geometry

`geom_line()` connects values in order along the x-axis, typically time.

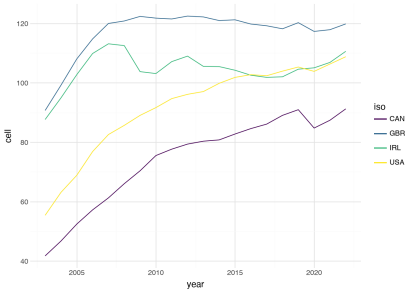
It takes the same aesthetics as `geom_point()`.

```
(
    cellphone
    .filter(c.iso.is_in(["USA", "GBR", "IRL", "CAN"]))
    .pipe(ggplot, aes(c.year, c.cell))
    .geom_line(aes(color=c.iso))
)
```

Four Countries, Four Lines

Each country traces its own line across years, colored by `iso`, sharing one x-axis and one y-axis.

Cellphone adoption rose steeply and at different rates across these four countries over the same two decades.



Cellphone subscriptions per 100 people, 2003–2022, for four countries.

Bar Geometry

`geom_col()` draws a bar per row, with length given by `y`.

`.coord_flip()` swaps the axes, handy for long category names.

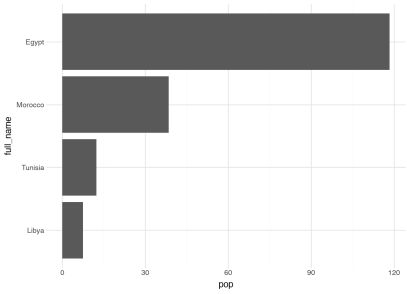
A categorical axis takes its order from the data, so `.sort()` first if order matters.

```
(
    country
    .filter(c.subregion == "Northern Africa")
    .sort(c.pop)
    .pipe(ggplot, aes(c.full_name, c.pop))
    .geom_col()
    .coord_flip()
)
```

Population by Country, Flipped

Sorting by `pop` first, then flipping the axes, puts the countries in a readable rank order instead of alphabetically.

Long category names — full country names here — fit naturally along a flipped y-axis.



Population of Northern African countries, sorted and flipped.

Multiple Datasets

Pass `data=` to a geometry to draw it from a different DataFrame than the rest of the plot.

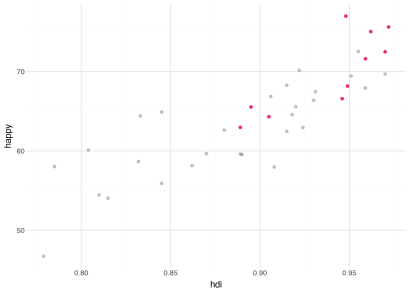
A common use is highlighting a subset on top of the full dataset.

```
country_ne = country.filter(c.subregion == "Northern Europe")
(
    country
    .filter(c.region == "Europe")
    .pipe(ggplot, aes(c.hdi, c.happy))
    .geom_point(alpha=0.2)
    .geom_point(color="#F5276C", data=country_ne)
)
```

Highlighting a Subset

The full set of European countries sits faded in the background (`alpha=0.2`); Northern European countries are drawn again, in pink, on top.

Both layers come from the same plot, but each `geom_point()` call reads from a different DataFrame.



Northern European countries highlighted against the full European scatter plot.

A Note on Labels

Every plot so far uses the raw column names as its axis labels. That is a deliberate default while exploring: it costs no extra effort, and the label can never drift out of sync with the column it names.

- `labs()` overrides any label: `x`, `y`, `title`, `subtitle`, legend titles
- Save the override for when a plot is ready to share, not while still exploring

Outline

Grouping and Aggregation

Every question in this chapter has two parts: split the rows into groups, then compute one number per group.

- `.group_by()` splits rows into groups, with syntax like `.sort()`
- `.agg()` computes one row of summaries per group
- Polars keeps these two steps strictly separate — understanding why is most of this chapter

Grouping Data

`.group_by()` groups rows by one or more columns; the method chained after it runs once per group.

Combined with `.sort()`, this is the standard way to pull the top row of each group — here, the largest country in each region.

```
(
    country
    .sort(c.pop, descending=True)
    .group_by(c.region)
    .head(n=1)
)
```

```
shape: (5, 15)
region      full_name                   pop
"Americas"  "United States of America"  347.3
"Oceania"   "Australia"                 26.97
"Europe"    "Russian Federation"        144.0
"Asia"      "India"                     1464
"Africa"    "Nigeria"                   237.5
```

What group_by Returns

`.group_by()` alone returns no data, just a `GroupBy` object: a promise that the next method should run once per group.

It must be consumed by a method like `.agg()`, `.head()`, or `.mean()`. There is no grouped DataFrame to save in a variable and reuse.

```
country.group_by(c.region)
# <polars.dataframe.group_by.GroupBy
#  object at 0x...>
```

Aggregation

`.agg()` computes a set of summaries per group, returning one row for each group.

Only the grouping column and the summaries you ask for survive; everything else is dropped.

```
(
    country
    .group_by(c.region)
    .agg(
        hdi_avg = c.hdi.mean()
    )
)
```

```
shape: (5, 2)
region      hdi_avg
"Africa"    0.5834
"Americas"  0.7836
"Europe"    0.8989
"Asia"      0.7609
"Oceania"   0.948
```

Several Summaries at Once

List several expressions inside `.agg()` to compute them all together.

`pl.len()` counts the rows in each group.

```
(
    country
    .group_by(c.region)
    .agg(
        hdi_avg = c.hdi.mean(),
        hdi_median = c.hdi.median(),
        count = pl.len()
    )
)
```

```
shape: (5, 4)
region      hdi_avg  hdi_median  count
"Africa"    0.5834   0.571       37
"Americas"  0.7836   0.786       19
"Europe"    0.8989   0.9115      38
"Asia"      0.7609   0.766       39
"Oceania"   0.948    0.948       2
```

Stable Group Order

Groups are built in parallel with a hash table, so their output order is not fixed from one run to the next.

`maintain_order=True` returns groups in the order their first row appears in the data — worth the small cost for a final table.

```
(
    country
    .group_by(c.region, maintain_order=True)
    .agg(
        hdi_avg = c.hdi.mean(),
        count = pl.len()
    )
)
```

```
shape: (5, 3)
region      hdi_avg  count
"Africa"    0.5834   37
"Americas"  0.7836   19
"Europe"    0.8989   38
"Asia"      0.7609   39
"Oceania"   0.948    2
```

Grouping by Multiple Columns

Pass several columns to `.group_by()`; a group is then every combination of values that actually occurs in the data.

```
(
    country
    .with_columns(north = c.lat >= 0)
    .group_by(c.region, c.north, maintain_order=True)
    .agg(
        hdi_avg = c.hdi.mean(),
        count = pl.len()
    )
)
```

```
shape: (8, 4)
region      north  hdi_avg  count
"Africa"    true   0.5604   23
"Americas"  true   0.7752   12
"Europe"    true   0.8989   38
"Asia"      true   0.7618   38
"Africa"    false  0.6211   14
"Americas"  false  0.798    7
"Oceania"   false  0.948    2
"Asia"      false  0.728    1
```

Aggregating Expressions

What you aggregate need not be a bare column — any expression works, built and reduced per group inside `.agg()`.

Below, `.sum()` on a Boolean expression counts how many rows in each group satisfy it.

```
(
    country
    .group_by(c.region, maintain_order=True)
    .agg(
        n_large = (c.pop > 100).sum()
    )
)
```

```
shape: (5, 2)
region      n_large
"Africa"    4
"Americas"  3
"Europe"    1
"Asia"      8
"Oceania"   0
```

Counting Rows vs. Values

`pl.len()` counts the rows in a group, regardless of missing data.

`.count()`, called on a column, counts only its non-missing values — smaller than `pl.len()` whenever that column has holes.

```
(
    country
    .group_by(c.region, maintain_order=True)
    .agg(
        n = pl.len(),
        n_gini = c.gini.count()
    )
)
```

```
shape: (5, 3)
region      n    n_gini
"Africa"    37   35
"Americas"  19   19
"Europe"    38   36
"Asia"      39   33
"Oceania"   2    1
```

Counting Distinct Values

`.n_unique()` counts the distinct values in a column — a different question from counting rows.

It treats `null` as a value of its own; drop nulls first if you want the count of distinct values actually present.

```
(
    country
    .select(
        n_gini = c.gini.n_unique(),
        n_gini_present =
            c.gini.drop_nulls().n_unique()
    )
)
```

```
shape: (1, 2)
n_gini  n_gini_present
101     100
```

Aggregating the Whole Frame

`.agg()` always belongs to a `.group_by()`. For one summary of the entire dataset, use `.select()` with the same aggregating expressions instead.

```
(
    country
    .select(
        pop_total = c.pop.sum(),
        hdi_avg = c.hdi.mean(),
        n = pl.len()
    )
)
```

```
shape: (1, 3)
pop_total  hdi_avg  n
7802.082   0.757    135
```

Aggregation as Row Reduction

Every aggregation reduces: it takes the many rows of a group and returns exactly one row for that group. Once a country's values are folded into the average, the row itself is gone.

- Fine when the answer you want is one number per group
- Not enough when you want a group-level number attached to every original row — e.g., each country's share of its region's population
- The next chapter, window functions, computes a value per group while keeping every row

Aggregation Functions

- `.mean()`, `.median()`, `.min()`, `.max()`, `.sum()`, `.quantile()`
- `.first()`, `.last()`, `.any()`, `.all()`
- `.n_unique()`, `.count()`, `.len()`

Each is called on a column or an expression, exactly like `c.hdi.mean()`.

```
(
    country
    .select(
        r = pl.corr(c.hdi, c.happy)
    )
)
```

```
shape: (1, 1)
r
0.7807
```

Outline

Window Functions

Aggregation answers questions about groups by collapsing each group to a single row. Window functions compute the same group-level quantities but attach the answer to every original row instead.

- **Aggregation reduces rows**; **window functions preserve them**
- `.over()` scopes a computation to each group, without collapsing the table
- Some window tools also depend on row order: shifting, ranking, running totals

The Problem

Suppose we want each country's share of its region's population.

`.agg()` gives the regional totals, but as a separate table with one row per region — lining it back up against each country would take a join.

```
(
    country
    .group_by(c.region)
    .agg(pop_total = c.pop.sum())
)
# one row per region
```

```
shape: (5, 2)
region      pop_total
"Americas"  974.1
"Oceania"   32.23
"Europe"    733.2
"Africa"    1330
"Asia"      4732
```

The over Method

`.over()` scopes an aggregation to each group, then writes that group's answer onto every row of the group — no summary table, no join.

Read it inside out: `c.pop.sum()` is a sum, `.over(c.region)` scopes it to each region.

```
(
    country
    .with_columns(
        pop_share = c.pop / c.pop.sum().over(c.region) * 100
    )
    .sort(c.pop_share, descending=True)
)
```

```
shape: (135, 16)
iso  full_name          region     pop_share
AUS  "Australia"        "Oceania"  83.7
USA  "United States"    "Americas" 35.65
IND  "India"            "Asia"     30.93
CHN  "China"            "Asia"     29.92
BRA  "Brazil"           "Americas" 21.85
...  ...                ...        ...
```

Same Number, Different Shape

`.agg()` collapses to one row per group.

`.with_columns()` with `.over()` keeps all 135 rows, with the regional average repeated on each.

```
country.group_by(c.region).agg(
    hdi_avg = c.hdi.mean()
)
# one row per region

country.with_columns(
    hdi_avg = c.hdi.mean().over(c.region)
)
# 135 rows, hdi_avg repeated per region
```

```
shape: (5, 2)
region      hdi_avg
"Asia"      0.7609
"Americas"  0.7836
"Europe"    0.8989
"Oceania"   0.948
"Africa"    0.5834
```

```
shape: (135, 4)
iso  region      hdi    hdi_avg
SEN  "Africa"    0.53   0.5834
VEN  "Americas"  0.709  0.7836
FIN  "Europe"    0.948  0.8989
USA  "Americas"  0.938  0.7836
LKA  "Asia"      0.776  0.7609
```

Windows Inside filter()

A windowed expression is still just an expression, so it can be used anywhere one is expected — including `.filter()`.

No new column is created; the regional medians are computed, compared, and discarded.

```
(
    country
    .filter(c.hdi > c.hdi.median().over(c.region))
)
```

```
shape: (66, 15)
iso  region       hdi
FIN  "Europe"     0.948
USA  "Americas"   0.938
LKA  "Asia"       0.776
SGP  "Asia"       0.946
...  ...          ...

# down from 135 rows: roughly half of
# each region falls below its median
```

Shift

`.shift(n)` moves a column's values down by `n` rows; negative `n` shifts upward.

Because "the row above" depends on sort order, `.shift()` should sit directly after the `.sort()` that gives it meaning.

```
(
    country
    .sort(c.pop, descending=True)
    .with_columns(
        perc_larger = c.pop / c.pop.shift(-1) * 100
    )
)
```

```
shape: (135, 16)
iso  full_name        pop     perc_larger
IND  "India"          1464    103.4
CHN  "China"          1416    407.8
USA  "United States"  347.3   121.5
IDN  "Indonesia"      285.7   112.0
...  ...              ...     ...
ISL  "Iceland"        0.398   null
```

Shift Within Groups

On a table with several countries stacked together, a plain `.shift()` bleeds across country boundaries.

`.over()` restarts the shift at each group: “look one row up, but never past this country’s first year.”

```
(
    cellphone
    .sort(c.iso, c.year)
    .with_columns(
        change = c.cell - c.cell.shift(1).over(c.iso)
    )
)
```

```
shape: (3480, 4)
iso  year  cell     change
AFG  2003  0.8798   null
AFG  2004  2.547    1.667
AFG  2005  4.917    2.37
AFG  2006  9.913    4.996
...  ...   ...      ...
ZWE  2021  90.25    5.293
ZWE  2022  88.996   -1.258
```

Rank

`.rank()` assigns each value its position within the column; `descending=True` makes rank 1 the largest.

Scoped with `.over()`, it ranks within each group instead of across the whole table.

```
(
    country
    .with_columns(
        hdi_rank = c.hdi.rank(descending=True)
                        .over(c.region)
    )
    .select(c.iso, c.region, c.hdi, c.hdi_rank)
    .sort(c.region, c.hdi_rank)
)
```

```
shape: (135, 4)
iso  region      hdi    hdi_rank
MUS  "Africa"    0.806  1.0
EGY  "Africa"    0.754  2.0
TUN  "Africa"    0.746  3.0
...  ...         ...    ...
AUS  "Oceania"   0.958  1.0
NZL  "Oceania"   0.938  2.0
```

Cumulative Sum

`.cum_sum()` gives a running total: each row holds the sum of every value up to and including itself.

Sorting first, then dividing by the plain `.sum()`, gives a cumulative share of the whole.

```
(
    country
    .sort(c.pop, descending=True)
    .select(
        c.iso,
        c.pop,
        cum_share = c.pop.cum_sum() / c.pop.sum() * 100
    )
)
```

```
shape: (135, 3)
iso  pop    cum_share
IND  1464   18.76
CHN  1416   36.91
USA  347.3  41.36
IDN  285.7  45.03
PAK  255.2  48.3
NGA  237.5  51.34
...  ...    ...
ISL  0.398  100.0
```

Rolling Windows

`.rolling_mean(window_size=)` replaces each row with the average of itself and its recent neighbors, smoothing out short-term noise.

The first `window_size - 1` rows are `null`, since they don't yet have a full window behind them.

```
(
    cellphone
    .filter(c.iso == "USA")
    .sort(c.year)
    .with_columns(
        cell_smooth = c.cell.rolling_mean(window_size=5)
    )
)
```

```
shape: (20, 4)
iso  year  cell    cell_smooth
USA  2003  55.41   null
USA  2004  63.12   null
USA  2005  68.88   null
USA  2006  76.86   null
USA  2007  82.59   69.37
USA  2008  85.68   75.43
...  ...   ...     ...
USA  2022  108.8   105.6
```

Aggregation Versus over

The choice comes down to one question: what should the result look like?

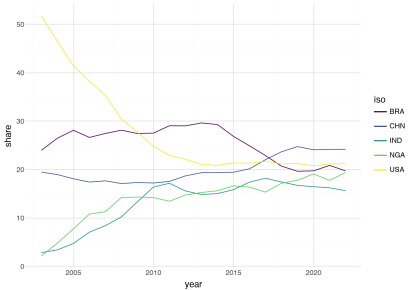
- Want a table of groups, one row each? Use `.group_by()` and `.agg()`
- Want the original table with group information written on it, or filtered by it? Use `.over()`
- The two pair naturally: `.agg()` to report a summary, `.over()` to flag what drives it

Windows in Both Directions

Every window so far grouped by country and looked across years. This one runs the other way: within each *year*, it computes each country's share of five countries' combined subscriptions.

Raw subscription counts all rise steeply, which makes trends hard to compare; shares put every country on a common scale.

This plot needs one point per country per year — exactly the rows an aggregated table would have collapsed away.



Each country's share of five countries' combined cellphone subscriptions, by year.

Outline

Statistical Geometries

A bar chart, a histogram, a boxplot: none of these draw one shape per row. Each one first runs a grouped computation over the data, then draws the result.

A statistical geometry is a `.group_by()` and `.agg()` in disguise — it counts, bins, or summarizes for you before it ever reaches the page.

Once you see that, the rest of this chapter is mostly control and polish: choosing scales, splitting a plot into panels, and labeling the result for someone else to read.

geom_bar Counts for You

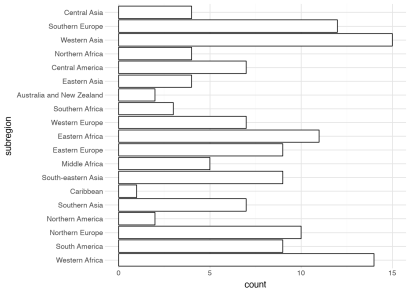
`geom_bar` takes one categorical aesthetic, groups the rows by its value, counts each group, and draws the counts as bars — the `.group_by()` and `.agg()` happen internally.

```
(
    country
    .pipe(ggplot, aes(c.subregion))
    .geom_bar(fill="white", color="black")
    .coord_flip()
)
```

One Bar per Subregion

Each bar's length is a count of rows, computed by `geom_bar` itself — no `.group_by()` anywhere in the code.

Subregions appear in the order they first occur in `country`, not sorted by count.



Number of countries per subregion.

geom_col Draws What You Give It

`geom_col` does no counting of its own; it takes an `x` and a `y` and draws each `y` as a bar's height.

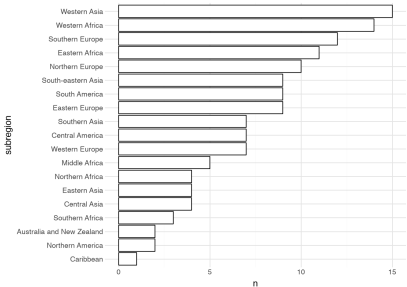
If you find yourself computing a count with `.group_by()` only to feed it to `geom_bar`, you wanted `geom_col`.

```
(
    country
    .group_by(c.subregion)
    .agg(pl.len().alias("n"))
    .sort(c.n)
    .pipe(ggplot, aes(c.subregion, c.n))
    .geom_col(fill="white", color="black")
    .coord_flip()
)
```

The Same Bars, Pre-Counted

Same bars as before, but this time `.group_by()` and `.agg()` compute the counts explicitly, then `.sort()` orders them by size.

`geom_col` just draws the `n` column it's handed — sorting the categories was our job, not the geometry's.



Number of countries per subregion, sorted by count.

Ordering Categories

A categorical axis follows the order values first appear in the data — there is no separate sort step inside the plot.

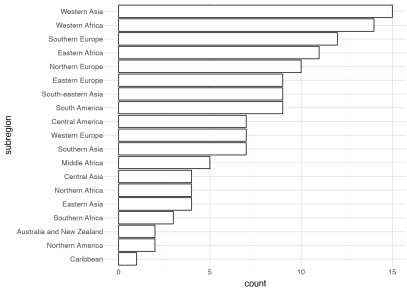
When the value to order by lives only inside the plot (like a bar's count), sort by the same computation written with `.over()` before plotting.

```
(
    country
    .sort(pl.len().over(c.subregion))
    .pipe(ggplot, aes(c.subregion))
    .geom_bar(fill="white", color="black")
    .coord_flip()
)
```

geom_bar, Sorted This Time

`geom_bar` still counts the rows itself, but sorting the *rows* by `pl.len().over(c.subregion)` first changes the category order it reads off.

Identical bars to the very first bar chart in this chapter, now ranked smallest to largest.



Number of countries per subregion, ordered by count via a pre-sort.

Categorical Axes

Plotnine reads a column of numbers as continuous, spacing it along an axis by distance rather than treating each value as its own group.

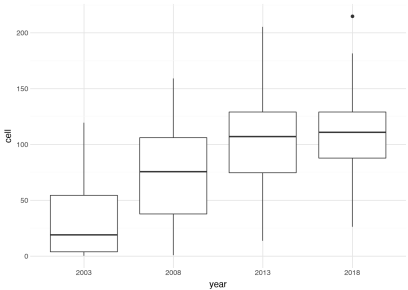
Casting a column to `pl.String` forces each distinct value to become its own category with its own box or bar.

```
(
    cellphone
    .filter(c.year.is_in(
        [2003, 2008, 2013, 2018]
    ))
    .sort(c.year)
    .with_columns(c.year.cast(pl.String))
    .pipe(ggplot, aes(c.year, c.cell))
    .geom_boxplot()
)
```

Four Years as Four Categories

Casting `year` to a string turns four numbers into four distinct categories, each with its own box.

Had `year` stayed numeric, plotnine would have spaced the boxes by distance along a continuous axis instead of treating them as groups.



Cellphone subscriptions by year, treated as a categorical axis.

Distributions

For a numeric column, a *group* means a bin. `geom_histogram` cuts the number line into equal intervals and counts how many values land in each one; `binwidth` and `boundary` control the cuts.

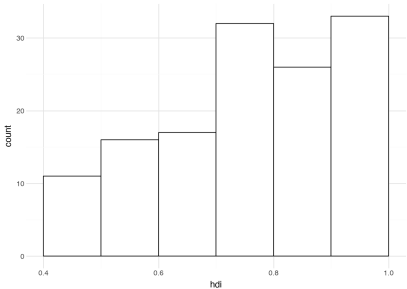
`geom_density` models the same distribution as a smooth curve instead of tallying it into bars.

```
(
    country
    .pipe(ggplot, aes(c.hdi))
    .geom_histogram(
        fill="white", color="black",
        binwidth=0.1, boundary=0
    )
)
```

One Distribution, Binned

Each bar spans a 0.1-wide slice of HDI, from a boundary at 0, and counts how many countries land inside it.

Most countries cluster in the upper HDI bins; only a handful fall below 0.5.

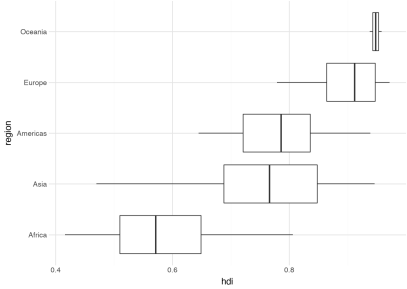


Distribution of HDI across all countries.

Boxplots as Grouped Quantiles

A boxplot combines both ideas above: it summarizes a numeric variable separately for each category. Under the hood it is three quantiles per group — 25th, 50th, 75th — plus whiskers and outlier points.

The regions here are ordered by `c.hdi.median().over(c.region)`, not by HDI itself — sorting by a country's own HDI would order regions by their *lowest* value, not their middle.



HDI distribution by region, ordered by median.

Fitting a Model

`geom_smooth` summarizes differently: instead of grouping rows, it fits a model to the relationship between two numeric variables and draws the fitted curve.

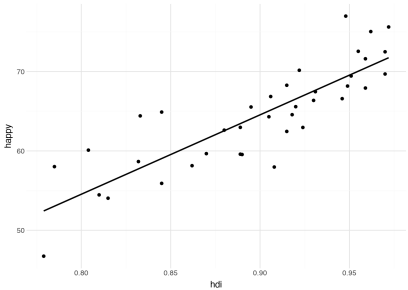
`method="lm"` fits a straight line; leaving `method` out fits a more flexible, non-linear curve.

```
(
    country
    .filter(c.region == "Europe")
    .pipe(ggplot, aes(c.hdi, c.happy))
    .geom_point()
    .geom_smooth(method="lm", se=False)
)
```

A Fitted Line

`geom_smooth` adds a second layer on top of the points: a straight line fit by least squares, summarizing the trend those points suggest.

`se=False` hides the shaded confidence band that plotnine draws around the line by default.



HDI versus happiness for European countries, with a fitted linear trend.

Scales

Every aesthetic has a scale attached to it, controlling how a value turns into something visible — axis range, tick labels, or which colors mark which categories.

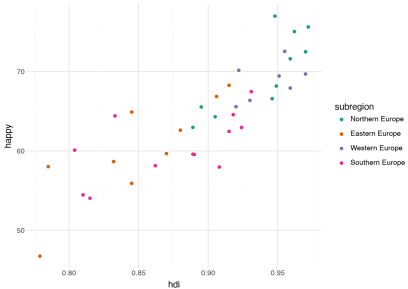
Add a scale function to change the default. `scale_color_brewer` swaps the viridis ramp for hand-picked, easy-to-distinguish colors — useful when categories have no natural order.

```
(
    country
    .filter(c.region == "Europe")
    .pipe(ggplot, aes(c.hdi, c.happy))
    .geom_point(aes(color=c.subregion))
    .scale_color_brewer(type="qual", palette=2)
)
```

A Different Color Scale

Same points, same mapping of `subregion` to color — only the palette generating the colors has changed.

Brewer's qualitative palettes are built for exactly this: categories with no inherent order, needing colors that are easy to tell apart.



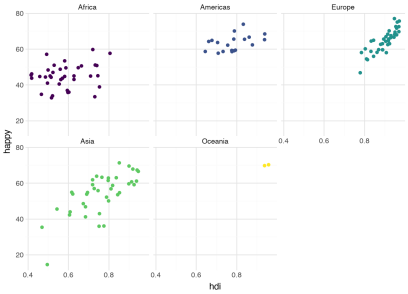
HDI versus happiness, colored by subregion using a Brewer qualitative palette.

Facets

`facet_wrap("region")` lays out a grid of small panels, one per region, instead of packing every group into a single plot.

Mapping color to the same grouping variable and hiding the legend lets color reinforce which panel holds which group.

Every panel shares one x-axis and one y-axis by default, which is what makes them comparable at a glance — `scales="free"` breaks that comparison and should be used rarely.



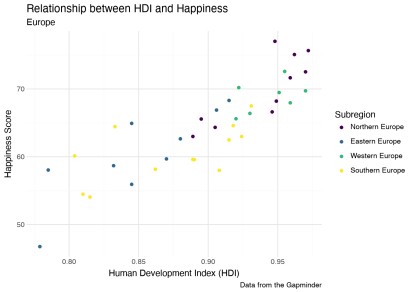
HDI versus happiness, faceted by region.

Labels and Themes

Default labels take no effort and use the exact column names you need elsewhere in a pipeline — the right choice while exploring.

Once a plot is meant for someone else, a `.labs()` layer adds real axis titles, a legend title, and a title/subtitle/caption for the plot as a whole.

Themes govern everything that isn't data: axis lines, ticks, background. `theme_minimal()` suits exploration; a sparer theme suits publication.



The same scatter plot, labeled for an audience.

Best Practices

- Bar plots, histograms, density plots: the distribution of one variable
- Scatter plots and boxplots: the relationship between two variables
- Scatter plots with text labels: individual points, not just the trend
- Use a variable color scale only after `x` and `y` are fully used
- Color and shape work only for a small set of categories — eight is a reasonable ceiling
- Line plots: a single quantity measured over time or another continuum

Outline

Combining Tables

A *primary key* is one or more columns that uniquely identify a row of a table — in `country`, the `iso` column.

A *foreign key* is a primary key from one table that appears in another. The same column, `iso`, is a foreign key wherever another table records which country a row belongs to.

The link between them is a *relation*, and a *join* is how we combine two tables along it — extending filtering and grouping to work across tables instead of just within one.

The Basic Join

`.join()` is a method of the table holding the foreign key (the “left” table); it takes the other table and the shared key column.

By convention, put the foreign-key table on the left and the primary-key table on the right — the output’s row count follows the left table.

```
(
    table_one
    .join(table_two, on=c.key_col)
)
```

Left Joins Keep Every Row

The default `how="inner"` drops any left-table row with no match on the right — an easy way to silently lose rows.

`how="left"` keeps every row of the left table, filling in `null` wherever the right table has no match.

```
(
    table_one
    .join(table_two, on=c.key_col, how="left")
)
```

Inner Joins Can Drop Rows

`border` has one row per pair of neighboring countries; `c_sml` has one row per country with a Gini coefficient.

Joining them on `iso` keeps only borders where the base country has economic data — 641 rows survive out of the original 829.

```
(
    border
    .join(c_sml, on=c.iso)
)
```

```
shape: (641, 3)
iso  iso_neighbor  gini
AFG  IRN           27.8
AFG  PAK           27.8
AFG  CHN           27.8
...  ...           ...
ZWE  ZAF           50.3
ZWE  ZMB           50.3
```

Restoring the Row Count

Switching to `how="left"` restores all 829 rows of `border`, filling `null` wherever `c_sml` had no matching `iso`.

Nothing about the match itself changed — only what happens to the rows that fail to match.

```
(
    border
    .join(c_sml, on=c.iso, how="left")
)
```

```
shape: (829, 3)
iso  iso_neighbor  gini
AFG  IRN           27.8
AFG  PAK           27.8
...  ...           ...
# 829 rows again —— unmatched iso
# values now carry a null gini
```

Different Key Names

When the shared key has different names in each table, use `left_on` and `right_on` instead of `on`.

This adds the *neighboring* country's Gini coefficient to each row, by matching `iso_neighbor` against `c_sml`'s `iso`.

```
(
    border
    .join(
        c_sml,
        left_on=c.iso_neighbor,
        right_on=c.iso
    )
)
```

```
shape: (667, 3)
iso  iso_neighbor  gini
AFG  IRN           40.9
AFG  PAK           29.6
AFG  CHN           38.2
AFG  TJK           34.0
...  ...           ...
```

```
# gini here is the NEIGHBOR's value --
# compare AFG-IRN: 40.9 vs. 27.8 above
```

Two Joins for Both Sides

Adding the base country's Gini *and* its neighbor's takes two joins chained one after the other, one per key.

`suffix="_neighbor"` renames the second join's columns so they don't collide with the first.

```
(
    border
    .join(c_sml, on=c.iso)
    .join(
        c_sml,
        left_on=c.iso_neighbor,
        right_on=c.iso,
        suffix="_neighbor"
    )
)
```

```
shape: (523, 4)
iso  iso_neighbor  gini  gini_neighbor
AFG  IRN           27.8  40.9
AFG  PAK           27.8  29.6
AFG  CHN           27.8  38.2
...  ...           ...   ...
```

Filtering Joins: Semi and Anti

A *semi-join* keeps left-table rows that have a match on the right; an *anti-join* keeps the ones that don't. Neither adds any columns — both just filter.

Here, every surviving row is an `iso` in `border` with no match anywhere in `country`, not just in the trimmed `c_sml`.

```
(
    border
    .join(country, on=c.iso, how="anti")
)
```

```
shape: (188, 2)
iso  iso_neighbor
AGO  COD
AGO  GAB
AGO  NAM
...  ...
VUT  SLB
WSM  USA
```

Diagnosing Bad Joins

Joining on a column that isn't actually unique in the right table silently multiplies rows: every match on the left pairs with *every* matching row on the right.

Check before you join, not after — comparing `.n_unique()` to `len()` on the key confirms the right table is safe to join against.

`bad_c_sml` is `c_sml` with Afghanistan's row deliberately duplicated, to show what the check catches.

```
(
    c_sml.select(c.iso).n_unique()
    == len(c_sml)
)
# True: iso is a genuine key in c_sml

(
    bad_c_sml.select(c.iso).n_unique()
    == len(bad_c_sml)
)
# False: bad_c_sml has a duplicated iso

border.join(c_sml, on=c.iso).shape
# (641, 3)

border.join(bad_c_sml, on=c.iso).shape
# (647, 3) — 6 extra rows, all
# duplicate copies of AFG's borders
```

join_asof: Nearest Match

`.join_asof()` matches each left row to the *nearest* value in the right table on a numeric column, rather than requiring exact equality.

Both tables must be pre-sorted by the join key. `strategy="nearest"` looks in both directions; the default, `backward`, looks only for earlier values.

```
(
    africa
    .join_asof(
        asia, on=c.gini,
        strategy="nearest",
        coalesce=False
    )
)
```

```
shape: (35, 4)
full_name    gini  full_name_right      gini_right
"Guinea"     29.6  "Pakistan"           29.6
"Egypt"      31.5  "Korea, Rep. of"     31.4
"Mauritania" 32.6  "Mongolia"           32.7
...          ...   ...                  ...
"S. Africa"  63.1  "Malaysia"           46.2
```

join_where: Conditional Joins

`.join_where()` takes a `.filter()`-style expression and keeps every pair of rows that satisfies it, with no equality requirement at all.

Use the `_right` suffix to refer to the second table's column when both sides share a name. This example pairs countries within 5 points of Gini.

```
(
    africa
    .join_where(
        asia,
        (c.gini - c.gini_right).abs() < 5
    )
)
```

```
shape: (414, 4)
full_name  gini  full_name_right         gini_right
"Guinea"   29.6  "Armenia"               25.2
"Guinea"   29.6  "United Arab Emirates"  26.0
"Guinea"   29.6  "Azerbaijan"            26.6
...        ...   ...                    ...
```

```
# 414 pairs out of 35x33 = 1155 possible
```

Attaching a Group Summary

A join is a second route to what `.over()` already does: attach a group-level number to every row without collapsing the table.

Aggregate to one row per region, then join that summary back on `region`. Reach for this version when you want the summary table itself, not just the joined column.

```
region_avg = (
    country
    .group_by(c.region)
    .agg(region_avg_gini = c.gini.mean())
)
(
    country
    .join(region_avg, on=c.region)
)
```

```
shape: (135, 16)
iso  region      region_avg_gini
SEN  "Africa"    41.92
VEN  "Americas"  45.12
FIN  "Europe"    30.85
USA  "Americas"  45.12
...  ...         ...
```

```
# identical to region_avg_gini from
# c.gini.mean().over(c.region) directly
```

Choosing a Join

- Adding columns by matching keys exactly? `.join()`, with `how` set to `inner`, `left`, `right`, or `full`
- Filtering one table by presence in another, no new columns? `how="semi"` or `how="anti"`
- Matching on the closest numeric value, not an exact one? `.join_asof()`
- Matching on an arbitrary condition between columns? `.join_where()`
- Attaching a group summary to every row? `.over()` is simplest; agg-then-join if you need the summary table too

Outline

Restructuring Data

Reshaping a table changes its *layout*, not its values — no information is added or removed, only rearranged.

Every layout is anchored to a *unit of observation*: the “who” or “what” that each row represents.

Picking the right layout, not the right values, is often what makes an analysis easy or painful.

Units of Observation

Tracking the heights of 100 plants over 14 days could be stored several ways:

- one row per plant, one column per day — unit of observation: *plant*
- one row per day, one column per plant — unit of observation: *day*
- one row per plant-and-day pair, 1,400 rows total — unit of observation: *plant* $\times$ *day*

None of these is wrong — each records the same measurements. Once a unit of observation is chosen, there is only one way to build the table.

Tidy Data

A table is *tidy* for a chosen unit of observation when:

- every variable is a single column
- every observation is a single row
- every table holds a single unit of observation

Tidiness is a property of a table *relative to* a unit of observation, not of the values themselves — a wide plant-by-day table is tidy for the unit *plant*, and untidy the moment the unit needed is *plant* $\times$ *day*.

A Tidy Table

`cellphone` has one row per country per year, so its unit of observation is *year* × *country*.

Each variable — `year`, `iso`, `cell` — gets its own column, so the table is already tidy for that unit.

`cellphone`

shape: (3480, 3)

iso	year	cell
AFG	2003	0.8798
AFG	2004	2.547
AFG	2005	4.917
AFG	2006	9.913
…	…	…
ZWE	2022	88.996

pivot(): Long to Wide

`.pivot()` needs three columns: `index` becomes the new unit of observation, `on` supplies the new column names, and `values` fills them in.

Combinations missing from the original table come back as `null`; chain `.fill_null(0)` if a default makes more sense.

```
cell_wide = (
    cellphone
    .pivot(
        index=c.year,
        on=c.iso,
        values=c.cell
    )
)
```

```
shape: (20, 175)
year  AFG      AGO      ALB    AND    ...
2003  0.8798   1.951    35.28  74.68  ...
2004  2.547    3.978    40.65  78.53  ...
2005  4.917    8.352    49.75  83.39  ...
...   ...      ...      ...    ...    ...

# 3 columns -> 175 columns, one per iso
```

unpivot(): Wide to Long

`.unpivot()` reverses a pivot, collapsing many columns back into two: `variable` and `value` by default.

Aside from row and column order, this recovers the original long table exactly — pivoting and unpivoting only rearrange values.

```
(
    cell_wide
    .unpivot(index=c.year)
)
```

```
shape: (3480, 3)
year  variable  value
2003  "AFG"     0.8798
2004  "AFG"     2.547
2005  "AFG"     4.917
...   ...       ...

# default names: variable, value
```

plotnine Needs a Column to Map

plotnine expects long data. In `cell_wide` there is no `iso` column, only a separate column per country, so there is nothing to map `color` to.

This runs, but only draws one hard-coded country — there is no way to get “one line per country” without a separate call for each column.

```
(
    ggplot(
        cell_wide,
        aes(x=c.year, y=c.usa)
    )
    .geom_line()
)
```

One Line per Country

Naming the new columns as they're built restores `iso` as a real column, so `color=c.iso` is enough to draw one line per country.

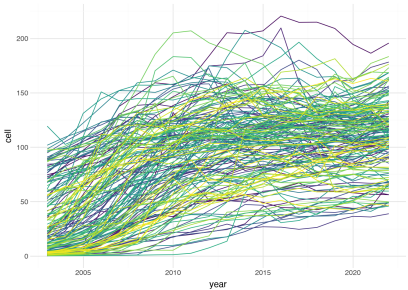
This is also why plotting `cellphone` directly always worked — it was already stored long, one row per year-and-country.

```
cell_long = (
    cell_wide
    .unpivot(
        index=c.year,
        variable_name=c.iso,
        value_name=c.cell
    )
)
(
    cell_long
    .pipe(
        ggplot,
        aes(x=c.year, y=c.cell,
            color=c.iso)
    )
    .geom_line()
)
```

Every Country, One Plot

With `iso` restored as a real column, `color=c.iso` draws all 175 countries at once — one `geom_line()` call, no loop.

The legend is hidden here since 175 entries wouldn't fit; the point is the shape of the data, not identifying any one line.



Cellphone subscriptions for every country, 2003–2022, one line each.

Splitting Text into List Columns

Sometimes multiple values are packed into one cell, usually after a cleaning step rather than as raw input.

`.str.split()` turns delimited text — here, the `lang` column's pipe-separated codes — into a list stored in each cell.

```
(
    country
    .select(c.full_name, c.lang)
    .with_columns(
        lang_codes =
            c.lang.str.split("|")
    )
)
```

```
shape: (135, 3)
full_name     lang          lang_codes
"Senegal"     "pbp|fra|wol" ["pbp","fra","wol"]
"Finland"     "fin|swe"     ["fin","swe"]
"USA"         "eng"         ["eng"]
...           ...           ...
```

explode(): One Row per Element

A list column is hard to work with directly. `.explode()` unravels it, giving each element its own row and duplicating the other columns to match.

Watch the row count — exploding multiple columns at once can make a table grow very large, very fast.

```
(
    country
    .select(c.full_name, c.lang)
    .with_columns(
        lang_codes =
            c.lang.str.split("|")
    )
    .explode(c.lang_codes)
)
```

```
shape: (265, 3)
full_name    lang           lang_codes
"Senegal"    "pbp|fra|wol"  "pbp"
"Senegal"    "pbp|fra|wol"  "fra"
"Senegal"    "pbp|fra|wol"  "wol"
...          ...            ...

# 135 rows -> 265 rows
```

Round Trips, Not Aggregation

- Pivot and unpivot are round trips — pivot to wide, unpivot back, and (row order aside) you have the original table
- Aggregation is not reversible — collapsing rows to a group summary discards the individual values for good
- Reshaping changes a table's shape; aggregation changes its unit of observation, in one direction only
- Store data long: a long table can always be pivoted into whatever wide shape an analysis needs, but an aggregated table cannot be un-aggregated

Outline

Collecting Data

Most of a data scientist's time goes into collecting and cleaning data — if it's collected in a clean format from the start, that work disappears.

Chapter 9 gave the theory: what makes a table tidy, and why. This chapter is the practice — the decisions you actually face when building a dataset from scratch.

What Tables?

The unit of observation, simple or compound, determines what tables you need.

- *prefer longer to wider*: a long format is easier to extend and check, and there is usually only one of them, versus many possible wide layouts
- *respect the unit of observation*: only record data that describes the whole unit — metadata about a `plant` belongs in its own table, not copied into every row of a `plant` × `day` table

Merge tables later, once the data is collected, if an analysis needs them together.

What Variables?

- *prefer numeric*: store a variable as a number whenever you can; put units or currency in a separate column
- *use standards*: for character variables, reach for an existing standard (ISO country codes, USPS state codes) before inventing your own
- *atomic cells*: one value per cell — never pack several values into one entry, even separated by commas

Include a primary key column whenever a natural one already exists in the data.

What Variable Names?

- *allowed characters*: lower-case letters, numbers, and underscores only — start with a letter, never a number or underscore
- *simple names*: skip metadata like units in the name unless needed to tell two columns apart; prefer short, full names over cryptic abbreviations

These rules are for variable *names* — spaces and capitals are fine inside the actual values.

Storing Data in Spreadsheets

- *A1*: data collection starts in the upper-left cell — column names in row one, data below
- *sheets as tables*: one table per sheet, named the way you'll call it as a Python object
- *just tables*: nothing outside the rectangle — notes go in an extra column or table, not the margins
- *minimal formatting*: bolding a header or sizing columns is fine; formatting must never be the only place data lives

Notes Belong in a Column

An out-of-print book has no page count or weight — those cells are simply left blank, the one cross-software way to mark a value missing.

An explanatory note goes in its own `notes` column rather than a comment bolted onto a cell, keeping the table rectangular.

	A	B	C	D	E	F
1	**book_title**	**author**	**pages**	**weight**	**notes**	
2	On the Subject of Cooking	Marcus Gavius Apicius	320	1107		
3	Dietary Principles	Hu Sihui			Not in press	
4	Joy of Cooking	Irma Rombauer	1200	2177		
5	Larousse Gastronomique	Prosper Montagné	1216	3220		
6	Le Guide Culinaire	Georges Escoffier	680	1995		
7	Mastering the Art of French Cooking, Vol. 1	Julia Child	684	1450		
8	Mastering the Art of French Cooking, Vol. 2	Julia Child	554	1270		
9	Salt, Fat, Acid, Heat	Samin Nosrat	480	1270		
10	Science in the Kitchen and the Art of Eating Well	Pellegrino Artusi	653	1200		
11	The Boston Cooking-School Cook Book	Fannie Merritt Farmer	640	816		
12	The English Huswife	Gervase Markham	384	450		
13						
14						
15						
16						
17						

A cookbook table with blank cells for missing data and a dedicated notes column.

A Second Table for a Second Unit

The cookbooks table's unit of observation is `book_title`; author birth details describe a different unit, `author`, so they get their own table.

The `author` column in the cookbooks table is a foreign key into this one — exactly the relation that made joins work in Chapter 8.

	A	B	C	D	E	F	G
1	**author**	**nationality**	**birth_year**	**birth_month**	**birth_day**	**notes**	
2	Fannie Merritt Farmer	US	1857	3	23		
3	Georges Escoffier	FR	1846	10	28		
4	Gervase Markham	UK	1568			Birth year is approximate	
5	Hu Sihui	ZH				Active from 1314 to 1330	
6	Irma Rombauer	US	1877	10	30		
7	Julia Child	US	1912	8	15		
8	Marcus Gavius Apicius	IT	1			Born in Rome, ca. 1 AD	
9	Pellegrino Artusi	IT	1820	8	4		
10	Prosper Montagné	FR	1865	11	14		
11	Samin Nosrat	US	1979	11	7		
12							
13							
14							
15							
16							
17							

An authors table, referenced by the cookbooks table through its author column.

Plain Text vs. Spreadsheet Files

Plain text — almost always CSV — is the best format for sharing and long-term storage: no formatting information, readable by anything, and it holds exactly one table.

While a dataset is still being collected and edited, an Excel file or Google Sheet can be the better choice: it holds several tables at once and avoids round-trip errors between formats. Load one with `pl.read_excel()`.

Export to CSV once collection and cleaning are finished.

Reading Imperfect Files

Most files you didn't create won't load cleanly with the defaults. A handful of `pl.read_csv()` arguments handle almost every case.

`separator` and `encoding` fix the wrong delimiter or garbled characters; `skip_rows` drops a title row; `null_values` catches placeholder codes like `"-99"`; `schema_overrides` stops a column like a ZIP code from losing its leading zero.

```
survey = pl.read_csv(
    "survey_responses.csv",
    separator=";",
    encoding="windows-1252",
    skip_rows=2,
    null_values=["NA", "N/A", "-99"],
    schema_overrides={
        "zip_code": pl.Utf8
    },
)
```

The Data Dictionary

A data dictionary documents each variable: its name, units, and what it means — along with any decisions made while collecting it.

It can be a plain text file or, as here, its own sheet in the workbook — often the first or last one.

	A	B	C	D	E	F
1	**variable**	**long_name**	**units**	**description**		
2	book_title	Book Title		Book name, given in English		
3	author	Author's name		Authors given and family name		
4	pages	Number of pages		Pages in currently available edition		
5	weight	Book weight	grams	Weight of currently available edition		
6	nationality	Author's Nationality		Using ISO two letter country codes (i.e., CA)		
7	birth_year	Author's birth year		As numeric variable		
8	birth_month	Author's birth month		As numeric variable		
9	birth_day	Author's birth day		As numeric variable		
10						
11						
12						
13						
14						
15						
16						
17						

A data dictionary describing each variable in the cookbooks dataset.

Special Considerations

- *dates*: either split into separate year/month/day columns, or use the ISO 8601 standard formats `YYYY-MM-DD` and `YYYY-MM-DD HH:MM:SS`
- *missing values*: a blank cell, always — character columns can also use a documented missing-value code if needed
- *text and multimedia*: store the file name (and path) as a column; the media file itself is the data
- *synthetic keys*: when no natural primary key exists, generate one — a UUID is a common choice

Collecting Data Well

The schema decisions in this chapter — which tables, which variables, which names — are cheapest to get right before a single row is entered.

Every tool from the rest of this book, from grouping to joining to reshaping, works best on data that started tidy. Time spent designing a dataset up front is time saved cleaning it up later.